

Command Line Fundamentals Cheat Sheet

26 tables · 239 concepts · Updated 2026-05-25 · cheatgrid.com

The command line (also known as shell, terminal, or CLI) is a text-based interface where users interact with their operating system by typing commands. Originating from Unix in the 1970s, it remains the **most efficient way to automate tasks**, manage systems, and perform complex operations that would be cumbersome with graphical interfaces. While graphical UIs are intuitive for beginners, the command line offers **unmatched power through composability** — the ability to chain simple commands into sophisticated workflows using pipes and redirection. Mastering command line fundamentals means understanding not just individual commands, but **how they communicate through streams** (stdin, stdout, stderr), enabling you to build pipelines where each tool does one thing well and passes its output to the next.

CONTENTS

Directory Navigation	2
File Operations	2
Viewing File Contents	3
Wildcards and Globbing	3
Redirection Operators	4
Pipes and Command Chaining	4
Text Processing	5
File Searching	5
File Permissions	6
Process Management	6
Environment Variables	7
Special Variables	7
Command Substitution and Expansion	8
Quoting and Escaping	9
Conditionals	9
Loops	10
Networking Commands	10
Archiving and Compression	11
Command History and Shortcuts	11
Subshells and Command Grouping	12
Arrays (Bash 4.0+)	12
Input and Output Builtins	13
Shell Script Safety Options	13
Signal Handling	14
Disk and System Monitoring	14
Declaring and Typing Variables	15
References	15

TABLE 1

Directory Navigation

Navigating the filesystem fluently is the first skill any CLI user must internalize. `pwd`, `cd`, and `ls` form the absolute core; `pushd` / `popd` add a directory stack so you can jump between multiple locations without retyping paths — essential in long sessions involving many directories.

Command	Example	Description
cd	<code>cd /var/log</code>	• Change directory • <code>~</code> = home, <code>-</code> = previous dir, <code>..</code> = parent
ls	<code>ls -lah</code>	• List directory contents • <code>-l</code> long format, <code>-a</code> hidden files, <code>-h</code> human-readable sizes
pwd	<code>pwd</code>	Print absolute path of current working directory
pushd	<code>pushd /var/log</code>	• Change to directory and push it onto the directory stack • returns to it with <code>popd</code>
popd	<code>popd</code>	Remove top entry from directory stack and cd to the new top
dirs	<code>dirs -v</code>	• Display the directory stack • <code>-v</code> shows index numbers
tree	<code>tree -L 2</code>	• Display directory structure as visual tree • <code>-L</code> limits depth

TABLE 2

File Operations

These commands create, copy, move, rename, delete, and inspect files — the basic CRUD of the filesystem. The `-r` flag (recursive), `-i` (interactive confirmation), and `-p` (preserve attributes or create parents) recur throughout; understanding them reduces the need to memorise each command separately.

Command	Example	Description
cp	<code>cp -r src/ dest/</code>	• Copy files/directories • <code>-r</code> recursive, <code>-i</code> interactive, <code>-a</code> preserve attributes
mv	<code>mv old.txt new.txt</code>	• Move or rename files • same command serves both purposes
rm	<code>rm -rf dir/</code>	• Remove files/directories • <code>-r</code> recursive, <code>-f</code> force without confirmation
mkdir	<code>mkdir -p path/to/dir</code>	• Make directory • <code>-p</code> creates all missing parent directories
touch	<code>touch file.txt</code>	Create empty file or update timestamp of existing file
ln	<code>ln -s target link</code>	• Create link • <code>-s</code> for symbolic (soft) link, omit <code>-s</code> for hard link
diff	<code>diff file1 file2</code>	• Show line-by-line differences between two files • <code>-u</code> for unified (patch-friendly) format
stat	<code>stat file.txt</code>	Display detailed file metadata : size, inode, permissions, timestamps
mktemp	<code>tmp=\$(mktemp)</code>	• Create a unique temporary file safely • avoids race conditions vs manual naming
file	<code>file archive.bin</code>	Determine file type by content inspection (not extension)
rmdir	<code>rmdir emptydir</code>	• Remove empty directory only • fails if any files exist inside

TABLE 3

Viewing File Contents

Quickly reading files without opening an editor is one of the most frequent CLI tasks. `cat` dumps everything at once; `less` provides a scrollable pager; `head` / `tail` show the edges — choose based on file size and which part you need.

Command	Example	Description
cat	<code>cat file.txt</code>	• Concatenate and display entire file • also joins multiple files to stdout
less	<code>less file.txt</code>	• Scrollable pager • <code>q</code> quit, <code>/</code> search, <code>G</code> end, <code>g</code> start — the preferred viewer for large files
head	<code>head -n 20 file.txt</code>	• Display first N lines • default 10
tail	<code>tail -f /var/log/syslog</code>	• Display last N lines • <code>-f</code> follows the file in real-time — essential for log monitoring
tac	<code>tac file.txt</code>	• Print file in reverse line order (<code>cat</code> backwards) • useful for processing log files newest-first
nl	<code>nl file.txt</code>	• Print file with line numbers prepended • <code>-b a</code> numbers all lines including blank ones
more	<code>more file.txt</code>	• Forward-only pager • prefer <code>less</code> — more featureful and supports backward scrolling

TABLE 4

Wildcards and Globbing

Shell globbing lets the shell expand patterns to matching filenames *before* the command even runs — it is the shell, not the command, doing the matching. This distinction explains why `ls *.txt` and `ls '*.txt'` behave completely differently.

Pattern	Example	Description
*	<code>ls *.txt</code>	• Matches any characters (including zero) • most common wildcard
?	<code>ls file?.txt</code>	Matches exactly one arbitrary character
[abc]	<code>ls [abc]*.txt</code>	Matches any single character listed in the set
[a-z]	<code>ls [0-9]*.log</code>	Matches any single character in the range
[!abc]	<code>ls [!a]*.txt</code>	Matches any character not in the set (negation)
{a,b}	<code>cp file.{txt,bak}</code>	• Brace expansion — generates multiple strings • not strictly globbing but shell-expanded
** (globstar)	<code>ls **/*.txt</code>	• Matches files recursively in all subdirectories • requires <code>shopt -s globstar</code> (Bash 4.0+)

TABLE 5

Redirection Operators

Every Linux command has three standard streams: `stdin` (0), `stdout` (1), and `stderr` (2). Redirection operators route these streams to files, other commands, or each other — mastering them turns output into data you can store, compare, and feed back into pipelines.

Operator	Example	Description
<code>></code>	<code>cmd > file.txt</code>	- Redirect <code>stdout</code> to file overwrites existing content
<code>>></code>	<code>cmd >> file.txt</code>	- Redirect <code>stdout</code> to file appends without overwriting
<code><</code>	<code>cmd < input.txt</code>	Redirect file contents as stdin to command
<code>2></code>	<code>cmd 2> error.log</code>	Redirect stderr (file descriptor 2) to file
<code>2>&1</code>	<code>cmd > all.log 2>&1</code>	- Redirect <code>stderr</code> to same destination as stdout - order matters — <code>></code> must come first
<code>&></code>	<code>cmd &> all.log</code>	Shorthand: redirect both stdout and stderr to file (Bash 4+)
<code><<<</code>	<code>cmd <<< "text"</code>	Here-string — pass a single string as <code>stdin</code> without a file
<code><<EOF</code>	<code>cmd <<EOF</code> <code>text</code> <code>EOF</code>	Here-document — pass multi-line text as <code>stdin</code> until the delimiter
<code>>/dev/null</code>	<code>cmd > /dev/null 2>&1</code>	Discard all output <code>/dev/null</code> silently swallows anything written to it

TABLE 6

Pipes and Command Chaining

Pipes connect the `stdout` of one command to the `stdin` of the next, enabling the Unix philosophy of composing small tools into powerful workflows. The logical operators (`&&` , `||` , `;`) control whether subsequent commands run based on exit status — this is how scripts implement error-aware flows.

Operator	Example	Description
<code> </code>	<code>cmd1 cmd2</code>	Pipe — connects <code>stdout</code> of <code>cmd1</code> to <code>stdin</code> of <code>cmd2</code>
<code>&&</code>	<code>cmd1 && cmd2</code>	Logical AND — <code>cmd2</code> runs only if <code>cmd1</code> exits 0 (success)
<code> </code>	<code>cmd1 cmd2</code>	Logical OR — <code>cmd2</code> runs only if <code>cmd1</code> exits non-zero (failure)
<code>;</code>	<code>cmd1 ; cmd2</code>	- Run commands sequentially <code>cmd2</code> runs regardless of <code>cmd1</code>'s exit status
<code>tee</code>	<code>cmd tee file.txt</code>	Split output to both a file and <code>stdout</code> , continuing the pipeline
<code>&</code>	<code>cmd &</code>	- Run command in background - shell returns immediately
<code> &</code>	<code>cmd1 & cmd2</code>	Pipe both stdout and stderr of <code>cmd1</code> to <code>stdin</code> of <code>cmd2</code> (Bash 4+)

TABLE 7

Text Processing

These stream-editing tools are the heart of shell text manipulation. Each does one thing: `grep` filters lines, `sed` transforms them, `awk` processes structured fields, `sort` / `uniq` deduplicate. Chaining them with pipes builds data-processing pipelines that rival dedicated ETL tools.

Command	Example	Description
grep	<code>grep -r "error" .</code>	• Search text using patterns • <code>-i</code> case-insensitive, <code>-v</code> invert match, <code>-r</code> recursive, <code>-n</code> show line numbers
sed	<code>sed 's/old/new/g' file</code>	• Stream editor • <code>s/old/new/g</code> substitutes all occurrences per line • <code>-i</code> edits file in-place
awk	<code>awk '{print \$1}' file</code>	• Pattern scanning and field processing • <code>\$1</code> , <code>\$2</code> are field numbers, <code>\$NF</code> is last field
sort	<code>sort -n -k2 file.txt</code>	• Sort lines • <code>-n</code> numeric, <code>-r</code> reverse, <code>-k</code> by column, <code>-u</code> unique
uniq	<code>sort file uniq -c</code>	• Remove adjacent duplicates (must sort first) • <code>-c</code> counts occurrences
cut	<code>cut -d: -f1 /etc/passwd</code>	• Extract columns from text • <code>-d</code> sets delimiter, <code>-f</code> selects fields
tr	<code>tr 'a-z' 'A-Z'</code>	• Translate or delete characters • converts lowercase to uppercase in this example
wc	<code>wc -l file.txt</code>	Count lines (<code>-l</code>), words (<code>-w</code>), or bytes (<code>-c</code>)
paste	<code>paste file1 file2</code>	• Merge lines from multiple files side by side (column-wise) • <code>-d</code> sets delimiter
join	<code>join file1 file2</code>	Join lines on a common field (like a SQL inner join on sorted files)
comm	<code>comm -12 <(sort f1) <(sort f2)</code>	• Compare two sorted files • <code>-1</code> suppress unique-to-f1, <code>-2</code> unique-to-f2, <code>-3</code> suppress common

TABLE 8

File Searching

Finding files by name, content, size, type, or age is a daily task. `find` is the most powerful and flexible tool — combine it with `-exec` or `xargs` to act on results. `type` is the most reliable way to discover which version of a command the shell will actually run.

Command	Example	Description
find	<code>find . -name "*.log" -mtime -7</code>	• Search by name, type, size, time • <code>-exec cmd {}
</code> runs command on each match
grep -r	<code>grep -r "TODO" . --include="*.py"</code>	• Search file contents recursively • use <code>--include</code> to restrict to file types
xargs	<code>find . -name "*.tmp" xargs rm</code>	• Build and execute commands from stdin • passes results as arguments in batches
locate	<code>locate filename</code>	• Fast search using pre-built database • run <code>updatedb</code> to refresh • less flexible than <code>find</code>
type	<code>type -a python</code>	Show all matches in PATH plus whether it is a builtin, function, or alias — more thorough than <code>which</code>
which	<code>which python</code>	• Show first executable found in PATH • only finds external commands

Command	Example	Description
whereis	<code>whereis bash</code>	Locate binary, source, and manual page for a command

TABLE 9

File Permissions

Every file has an owner, a group, and permission bits for owner, group, and others (read=4, write=2, execute=1). Understanding `chmod` notation (both octal and symbolic), `umask`, and `chown` is essential for both security and scripting. Run `ls -l` to see the resulting permission string.

Command	Example	Description
chmod	<code>chmod 755 script.sh</code>	- Change file mode (permissions) - octal (<code>755</code>) or symbolic (<code>u+x</code> , <code>go-w</code>) notation
chown	<code>chown user:group file</code>	- Change file owner and group <code>-R</code> applies recursively
chgrp	<code>chgrp staff file.txt</code>	Change file group ownership only
umask	<code>umask 022</code>	- Set default permissions mask for newly created files - subtracts from 666 (files) / 777 (dirs)

TABLE 10

Process Management

Understanding how processes start, stop, pause, and run in the background is essential for system administration and scripting. Job control (`bg` , `fg` , `jobs`) manages processes within your terminal session; `nice` / `renice` control CPU priority; `nohup` detaches processes from the terminal so they survive disconnect.

Command	Example	Description
ps	<code>ps aux</code>	List running processes <code>a</code> all users, <code>u</code> user-oriented, <code>x</code> include non-terminal
kill	<code>kill -9 1234</code>	Send signal to process by PID <code>-15</code> SIGTERM (graceful), <code>-9</code> SIGKILL (force, cannot be caught)
top	<code>top</code>	- Display real-time system processes <code>q</code> quit, <code>k</code> kill, <code>r</code> renice
htop	<code>htop</code>	Interactive process viewer — more user-friendly than top with colour and mouse support
jobs	<code>jobs -l</code>	List background/suspended jobs in current shell session
bg	<code>bg %1</code>	- Resume suspended job in background <code>%N</code> refers to job number
fg	<code>fg %1</code>	Bring background job to foreground
Ctrl+Z	(keyboard)	Suspend current foreground process (SIGTSTP) - resume with <code>bg</code> or <code>fg</code>
nohup	<code>nohup cmd &</code>	- Run command immune to hangup - continues after terminal close - output goes to <code>nohup.out</code>
nice	<code>nice -n 10 cmd</code>	- Start command with lower CPU priority (niceness 10) - range -20 (highest) to 19 (lowest)
renice	<code>renice -n 5 -p 1234</code>	Change CPU priority of a running process by PID
wait	<code>wait \$pid</code>	Wait for background process to finish and capture its exit status

Command	Example	Description
timeout	<code>timeout 30 cmd</code>	- Run command with a time limit - exits 124 if the command times out
watch	<code>watch -n 2 df -h</code>	- Execute a command periodically (default 2s) and display output <code>-n</code> sets interval
pkill	<code>pkill -u username</code>	- Kill processes matching a pattern or user - more flexible than <code>killall</code>
killall	<code>killall firefox</code>	Kill all processes matching the executable name

TABLE 11

Environment Variables

Environment variables pass configuration into processes and scripts. `export` makes a variable available to child processes; `source` executes a script *in the current shell* so its variables and functions become part of your session. `PATH` is the most critical variable — it determines which commands the shell finds.

Command	Example	Description
export	<code>export PATH="\$PATH:/new/path"</code>	- Set environment variable for child processes - without <code>export</code>, variable is local to shell
source	<code>source ~/.bashrc</code>	- Execute script in current shell (not a subshell) - aliases and variables persist after it finishes
alias	<code>alias ll='ls -lah'</code>	- Create a command shortcut - put in <code>~/.bashrc</code> to make permanent
unalias	<code>unalias ll</code>	Remove an alias <code>unalias -a</code> removes all aliases
env	<code>env</code>	Display all environment variables for the current process
printenv	<code>printenv HOME</code>	- Print specific variable value - exits non-zero if variable is not set
echo \$VAR	<code>echo \$PATH</code>	Display variable value using parameter expansion
unset	<code>unset TEMP_VAR</code>	Remove variable from the environment

TABLE 12

Special Variables

Bash sets several automatic variables that capture process state after every command. `$?` after every command tells you whether it succeeded. `$@` and `$*` both expand all positional parameters but behave differently under double-quotes — `$@` preserves individual arguments, `$*` joins them with IFS.

Variable	Example	Description
\$?	<code>echo \$?</code>	Exit status of last command 0 = success, non-zero = failure
\$\$	<code>echo \$\$</code>	Process ID (PID) of current shell
#!	<code>echo \$!</code>	PID of last background process
\$0	<code>echo \$0</code>	Name of the script or shell being executed
\$1, \$2, ...	<code>echo \$1</code>	Positional parameters — command-line arguments passed to script
\$#	<code>echo \$#</code>	Count of positional parameters
\$@	<code>for arg in "\$@"</code>	All positional parameters as separate words (preserves argument boundaries under quotes)

Variable	Example	Description
<code>\$*</code>	<code>echo "\$@"</code>	All positional parameters as single string joined by first char of IFS
<code>\$LINENO</code>	<code>echo \$LINENO</code>	- Current line number in the script - useful in error messages
<code>\$BASHPID</code>	<code>echo \$BASHPID</code>	PID of the current Bash process (differs from <code>\$\$</code> inside subshells)

TABLE 13

Command Substitution and Expansion

Parameter expansion is one of the most powerful Bash features for manipulating strings, applying defaults, and extracting substrings — all without spawning a subprocess. Learning the full set of `${var...}` forms eliminates many cases where people reach for `sed` or `awk` to do simple string work.

Syntax	Example	Description
<code>\$(cmd)</code>	<code>now=\$(date)</code>	Command substitution — executes <code>cmd</code> and returns its stdout - preferred over backticks
<code>\$((expr))</code>	<code>echo \$((5 + 3))</code>	Arithmetic expansion — evaluates integer expression
<code>\${var}</code>	<code>echo \${name}</code>	Parameter expansion — unambiguous variable reference
<code>\${#var}</code>	<code>echo \${#str}</code>	String length of variable
<code>\${var:-default}</code>	<code>echo \${USER:-guest}</code>	- Use default if variable is unset or null - does not set variable
<code>\${var:=default}</code>	<code>\${PORT:=8080}</code>	Assign default if variable is unset or null - also sets the variable
<code>\${var:?error}</code>	<code>\${FILE:?File required}</code>	Exit with error message if variable is unset or null - enforces required parameters
<code>\${var:offset:length}</code>	<code>echo \${name:0:3}</code>	Substring extraction - offset is 0-based
<code>\${var#pattern}</code>	<code>echo \${path#*/}</code>	Remove shortest match from beginning
<code>\${var##pattern}</code>	<code>echo \${path##*/}</code>	Remove longest match from beginning (get filename from path)
<code>\${var%pattern}</code>	<code>echo \${file%.txt}</code>	Remove shortest match from end (strip extension)
<code>\${var/pattern/repl}</code>	<code>echo \${str/old/new}</code>	Replace first occurrence of pattern
<code>\${var//pattern/repl}</code>	<code>echo \${str//old/new}</code>	Replace all occurrences of pattern
<code>\${var^^}</code>	<code>echo \${name^^}</code>	Convert variable to uppercase (Bash 4.0+)
<code>\${var,,}</code>	<code>echo \${name,,}</code>	Convert variable to lowercase (Bash 4.0+)
<code>\${!var}</code>	<code>echo \${!prefix_name}</code>	Indirect expansion — treat value of <code>var</code> as the name of another variable
<code>`cmd`</code>	<code>now=`date`</code>	- Old-style backtick command substitution - non-nestable — prefer <code>\$()</code> (legacy)

TABLE 14

Quoting and Escaping

Quoting determines which characters the shell interprets literally versus specially. Single quotes are the most aggressive: nothing inside expands. Double quotes allow variable and command substitution. Getting quoting wrong is the single most common source of subtle bugs in shell scripts — always quote variables unless you deliberately want word-splitting.

Syntax	Example	Description
'...'	<code>echo 'Cost: \$10'</code>	• Single quotes — preserves literal value of all characters • no expansion of any kind
"..."	<code>echo "User: \$USER"</code>	• Double quotes — allows variable expansion and command substitution • spaces preserved
\	<code>echo \ \$USER</code>	Backslash — escapes next character to literal value
\$'...'	<code>echo \$'line1\nline2'</code>	ANSI-C quoting — interprets escape sequences like <code>\n</code> , <code>\t</code> , <code>\a</code>

TABLE 15

Conditionals

Shell conditionals test file attributes, string values, and numeric relationships. Prefer `[[ ... ]]` over `[ ... ]` in Bash scripts — it avoids word-splitting pitfalls and supports pattern matching and `&&` / `||` directly. The `case` statement is cleaner than chained `elif` when matching a single value against multiple patterns.

Operator	Example	Description
if [test]	<code>if [-f file]; then</code>	• POSIX test using <code>[</code> command • space required after <code>[</code> and before <code>]</code>
[if [[test]] (https://www.gnu.org/software/bash/manual/html_node/Conditional-Constructs.html)	<code>if [[\$a == \$b]]; then</code>	• Bash extended test • supports pattern matching, <code>&&</code> , <code>
</code>• <code>
</code>• directly, no word-splitting
case	<code>case "\$1" in start) start_svc ;; stop) stop_svc ;;) echo "unknown" ;; esac</code>	• Match variable against multiple patterns • cleaner than nested <code>elif</code> • <code>*</code> is the default catch-all
-f file	<code>[-f file.txt]</code>	True if path exists and is a regular file
-d dir	<code>[-d /tmp]</code>	True if path exists and is a directory
-e path	<code>[-e path]</code>	True if path exists (any file type)
-r / -w / -x	<code>[-x script.sh]</code>	True if file is readable / writable / executable by current user
-s file	<code>[-s file.txt]</code>	True if file exists and has non-zero size
-z string	<code>[-z "\$var"]</code>	True if string length is zero (empty or unset)
-n string	<code>[-n "\$var"]</code>	True if string length is non-zero
==, !=	<code>[[\$str == "text"]]</code>	• String comparison • <code>==</code> equality, <code>!=</code> inequality • <code>==</code> supports glob patterns in <code>[[]]</code>
-eq, -ne, -lt, -gt	<code>[\$a -eq 5]</code>	• Numeric comparison inside <code>[]</code> • equal, not-equal, less-than, greater-than

TABLE 16

Loops

Loops automate repetition. `for` iterates over a list or a C-style range; `while` runs until a condition is false; `until` is the inverse. The `while IFS= read -r line` pattern is the canonical way to process file lines safely — preserving whitespace and backslashes that naive loops would mangle.

Loop	Example	Description
for item in list	<pre>for i in 1 2 3; do echo \$i done</pre>	Iterate over list of items (words, filenames, or glob expansions)
for ((i=0; i<5; i++))	<pre>for ((i=0; i<5; i++)); do echo \$i done</pre>	C-style arithmetic for loop
while IFS= read -r line	<pre>while IFS= read -r line; do echo "\$line" done < file.txt</pre>	• Safe line-by-line file processing • preserves leading spaces and backslashes
while [test]	<pre>while [\$i -lt 10]; do ((i++)) done</pre>	Loop while condition is true
until [test]	<pre>until [\$i -ge 10]; do ((i++)) done</pre>	Loop until condition becomes true (opposite of while)
select item in list	<pre>select opt in start stop quit; do echo "Chose: \$opt" done</pre>	• Generate an interactive numbered menu • reads user selection into <code>REPLY</code>
break	<pre>break</pre>	Exit loop immediately
continue	<pre>continue</pre>	Skip to next iteration

TABLE 17

Networking Commands

Network tools handle the full stack from raw connectivity checks (`ping`) through file transfer (`curl` , `rsync` , `scp`) to port inspection (`ss` , `nc`). The modern `ip` command has replaced `ifconfig` and `route` on most Linux distributions; `ss` has replaced `netstat` .

Command	Example	Description
curl	<pre>curl -fsSL https://api.example.com</pre>	• Transfer data via HTTP, FTP, and many other protocols • <code>-fsSL</code> = fail silently, follow redirects
ssh	<pre>ssh user@host</pre>	• Secure remote login • <code>-i key.pem</code> specifies key, <code>-L</code> local port forwarding
rsync	<pre>rsync -av src/ dest/</pre>	• Efficient sync — only transfers differences • <code>-a</code> archive (preserves attrs), <code>-v</code> verbose
scp	<pre>scp file user@host:/path</pre>	Secure copy files over SSH
wget	<pre>wget https://file.com/data.zip</pre>	• Download files from the web • supports recursive download and resuming
ping	<pre>ping google.com</pre>	Test network connectivity with ICMP echo packets
ss	<pre>ss -tuln</pre>	• Socket statistics — modern replacement for <code>netstat</code> • <code>-t</code> TCP, <code>-u</code> UDP, <code>-l</code> listening, <code>-n</code> numeric
ip	<pre>ip a</pre>	• Modern replacement for <code>ifconfig</code> / <code>route</code> • <code>ip a</code> show addresses, <code>ip r</code> show routes

Command	Example	Description
nc	<code>nc -zv host 80</code>	• Netcat — raw TCP/UDP tool • <code>-z</code> port scan mode, also used to transfer files and test connectivity
dig	<code>dig example.com A</code>	• DNS lookup tool • more detailed output than <code>nslookup</code> • <code>+short</code> for concise results
traceroute	<code>traceroute google.com</code>	Trace the network route (hops) to a host
netstat	<code>netstat -tuln</code>	• Network statistics — shows listening ports • prefer <code>ss</code> on modern systems (legacy)

TABLE 18

Archiving and Compression

`tar` bundles files into archives; compression algorithms (`gzip`, `bzip2`, `xz`, `zstd`) shrink them. These are two separate concerns: `tar` flags `-z`, `-j`, `-J`, `--zstd` each invoke a different compressor. Cross-platform portability often dictates using `zip`.

Command	Example	Description
tar	<code>tar -czf archive.tar.gz dir/</code>	• Create/extract archives • <code>-c</code> create, <code>-x</code> extract, <code>-z</code> <code>gzip</code>, <code>-j</code> <code>bzip2</code>, <code>-J</code> <code>xz</code>, <code>-f</code> file
zip	<code>zip archive.zip files</code>	• Create cross-platform ZIP archive • <code>-r</code> recursive
unzip	<code>unzip archive.zip</code>	• Extract ZIP archive • <code>-l</code> list contents without extracting
gzip	<code>gzip file.txt</code>	• Compress using gzip • creates <code>file.txt.gz</code> and removes original
gunzip	<code>gunzip file.txt.gz</code>	Decompress gzip file
bzip2	<code>bzip2 file.txt</code>	• Compress using bzip2 • better compression ratio than <code>gzip</code>, slower
xz	<code>xz file.txt</code>	Compress using xz — best compression ratio of the common formats
zstd	<code>zstd file.txt</code>	• Compress using Zstandard — fast compression with ratio comparable to <code>xz</code> • widely used since 2023

TABLE 19

Command History and Shortcuts

Readline keyboard shortcuts and Bash history expansion dramatically reduce keystrokes. `Ctrl+R` reverse-incremental-search is the single most useful shortcut to memorise. History expansion (`!!`, `!$`, `!:n`) lets you reuse and modify previous commands without retyping.

Shortcut	Example	Description
Ctrl+R	(keyboard)	• Reverse search through command history • type to filter, Enter to run
!!	<code>sudo !!</code>	Repeat last command — canonical use: <code>sudo !!</code> to re-run with elevated privileges
!\$	<code>vim !\$</code>	Last argument of the previous command
Alt+.	(keyboard)	Insert last argument of previous command (interactive equivalent of <code>!\$</code>)
!n	<code>!42</code>	Execute command number n from history
!string	<code>!ssh</code>	Execute most recent command starting with string
!*	<code>rm !*</code>	All arguments from previous command

Shortcut	Example	Description
!:n	<code>!:2</code>	Nth argument from previous command (0 = command itself)
Ctrl+A	(keyboard)	Move cursor to beginning of line
Ctrl+E	(keyboard)	Move cursor to end of line
Ctrl+U	(keyboard)	Delete from cursor to beginning of line
Ctrl+K	(keyboard)	Delete from cursor to end of line
Ctrl+W	(keyboard)	Delete word before cursor
Ctrl+L	(keyboard)	Clear screen (equivalent to <code>clear</code>)
history	<code>history 20</code>	Display last N commands from command history

TABLE 20

Subshells and Command Grouping

Parentheses create a subshell — commands run in an isolated copy of the environment, so directory changes and variable assignments don't leak back to the parent. Curly braces group commands in the current shell. Process substitution creates anonymous pipe endpoints that look like files, enabling powerful comparisons and multi-source reads.

Syntax	Example	Description
(cmd1; cmd2)	<code>(cd /tmp; ls)</code>	- Execute commands in subshell - directory/variable changes don't affect parent
{ cmd1; cmd2; }	<code>{ echo "a"; echo "b"; }</code>	- Group commands in current shell - trailing semicolon and spaces around braces required
<(cmd)	<code>diff <(ls /dir1) <(ls /dir2)</code>	Input process substitution — treats command output as a readable file descriptor
>(cmd)	<code>cmd tee >(gzip > out.gz)</code>	Output process substitution — treats a command's stdin as a writable file descriptor

TABLE 21

Arrays (Bash 4.0+)

Bash arrays store ordered (indexed) or key-value (associative) collections. `mapfile` / `readarray` is the canonical way to read command output or file lines into an indexed array — it avoids the pitfalls of parsing with a `while` loop and is faster for large inputs. Arrays do not survive `export` .

Operation	Example	Description
Indexed array	<code>arr=(a b c)</code>	- Create indexed array - elements accessed by numeric index starting at 0
Associative array	<code>declare -A map</code> <code>map[key]=value</code>	- Create key-value (hash) array - elements accessed by string key - requires <code>declare -A</code>
\${arr[i]}	<code>echo \${arr[0]}</code>	Access element at index i
\${arr[@]}	<code>echo "\${arr[@]}"</code>	Expand all elements as separate words (preserves spaces in elements)
\${#arr[@]}	<code>echo \${#arr[@]}</code>	Get array length (number of elements)
arr+=(elem)	<code>arr+=(new)</code>	Append element to end of array
\${arr[@]:n:m}	<code>echo "\${arr[@]:1:3}"</code>	Array slice — elements from index n, for m elements
unset arr[i]	<code>unset arr[2]</code>	Delete element at index i (leaves gap) - does not re-index

Operation	Example	Description
mapfile	<code>mapfile -t lines < file.txt</code>	• Read lines from stdin/file into indexed array (<code>-t</code> strips trailing newlines) • also <code>readarray</code>

TABLE 22

Input and Output Builtins

The `read` builtin reads a line from stdin and assigns words to variables — it is the standard way to accept user input and to safely iterate over file lines. `printf` formats output with far more precision and reliability than `echo`, especially when output must be consistent across different systems.

Command	Example	Description
read	<code>read -r name</code>	• Read line from stdin into variable • <code>-r</code> disables backslash interpretation — always use it
IFS= read -r	<code>IFS= read -r line</code>	• Read line preserving leading/trailing whitespace • canonical safe form for line processing
read -p	<code>read -rp "Enter name: " name</code>	• Display prompt and read input • combines prompt and read in one step
read -s	<code>read -rsp "Password: " pass</code>	• Read input in silent mode (no echo) • use for passwords
read -t	<code>read -t 5 -r input</code>	• Read with timeout in seconds • exits non-zero if timeout expires
read -a	<code>read -ra words <<< "a b c"</code>	• Read words into an array • each whitespace-separated token becomes an element
printf	<code>printf "%-20s %5d\n" "\$name" \$num</code>	• Formatted output with C-style specifiers • <code>%s</code> string, <code>%d</code> integer, <code>%f</code> float, <code>%x</code> hex
printf -v	<code>printf -v result "%05d" 42</code>	• Store formatted output in variable instead of printing • avoids subshell overhead

TABLE 23

Shell Script Safety Options

`set` options harden scripts against common failure modes that Bash otherwise silently ignores. Running with `set -euo pipefail` catches errors immediately, treats unset variables as errors, and propagates non-zero exit codes through pipelines — these three options together are the standard guard for production scripts.

Option	Example	Description
set -e	<code>set -o errexit</code>	• Exit immediately on any command failure • prevents cascading errors from being silently ignored
set -u	<code>set -o nounset</code>	• Treat unset variables as errors • catches typos in variable names
set -o pipefail	<code>set -o pipefail</code>	• Pipeline exit status is the last non-zero exit code • without it, <code>cmd
</code> • <code>true</code> always succeeds
set -x	<code>set -o xtrace</code>	• Print each command before executing it • invaluable for debugging scripts
set -euo pipefail	<code>set -euo pipefail</code>	Combined idiom — the standard opening line for robust production scripts
shopt -s globstar	<code>shopt -s globstar</code>	Enable <code>**</code> to match files recursively across directories
shopt -s nullglob	<code>shopt -s nullglob</code>	• Unmatched glob patterns expand to empty string instead of literally • prevents <code>[: *.txt: unexpected</code> errors

Option	Example	Description
shopt -s extglob	<code>shopt -s extglob</code>	• Enable extended glob patterns like <code>!(*.txt)</code> , <code>+(a
• b)</code> , <code>*(pattern)</code>

TABLE 24

Signal Handling

Signals are asynchronous notifications sent to a process by the OS or another process. The `trap` command intercepts them and runs a handler — most commonly used with `EXIT` to guarantee cleanup code runs regardless of how the script exits. `SIGKILL` (9) cannot be trapped or ignored; `SIGTERM` and `SIGINT` can.

Command	Example	Description
trap 'cmd' SIGNAL	<code>trap 'rm -f /tmp/lock' EXIT</code>	• Run command when signal is received • <code>EXIT</code> fires on any exit (normal, error, or signal)
trap cleanup EXIT	<code>trap cleanup_fn EXIT SIGTERM</code>	• Handle multiple signals with same handler • use a function for complex cleanup
trap - SIGNAL	<code>trap - SIGINT</code>	Reset signal to its default action
trap " SIGNAL	<code>trap '' SIGINT</code>	Ignore signal completely (empty string command)
trap -p	<code>trap -p SIGINT</code>	• Display current traps • without argument shows all traps
kill -l	<code>kill -l</code>	List all signal names and numbers on the system
SIGTERM (15)	<code>kill -15 \$pid</code>	• Graceful shutdown signal • can be caught and handled • sent by <code>kill</code> with no flags
SIGINT (2)	<code>trap 'echo bye' SIGINT</code>	• Keyboard interrupt — generated by Ctrl+C • can be caught
SIGKILL (9)	<code>kill -9 \$pid</code>	• Force kill — cannot be caught, blocked, or ignored • use as last resort
SIGHUP (1)	<code>trap reload SIGHUP</code>	• Sent when terminal closes • traditionally used to tell daemons to reload config

TABLE 25

Disk and System Monitoring

`df` shows free space at the filesystem level; `du` drills into directory and file sizes. Together they answer where disk space went. `free` , `uname` , and `uptime` give a quick system health snapshot without needing a full monitoring tool.

Command	Example	Description
df	<code>df -h</code>	• Show disk space usage of all mounted filesystems • <code>-h</code> human-readable, <code>-T</code> show filesystem type
du	<code>du -sh /var/log</code>	• Show disk usage of a directory (recursive total) • <code>-s</code> summary, <code>-h</code> human-readable
du --max-depth	<code>du -h --max-depth=1 /</code>	• Show sizes of immediate subdirectories only • pipe to <code>sort -hr</code> to find largest
free	<code>free -h</code>	• Display memory usage (RAM + swap) • <code>-h</code> human-readable
uname	<code>uname -a</code>	• Print system information — kernel version, hostname, architecture • <code>-r</code> just kernel release
uptime	<code>uptime</code>	Show how long the system has been running and load averages (1, 5, 15 min)
whoami	<code>whoami</code>	Print current username

Command	Example	Description
id	<code>id</code>	Print user ID, group ID, and all group memberships for current user
hostname	<code>hostname -f</code>	- Print system hostname <code>-f</code> fully-qualified domain name
date	<code>date '+%Y-%m-%d %H:%M:%S'</code>	Display or format the current date and time using strftime format strings
lsuf	<code>lsuf -i :8080</code>	- List open files and network connections <code>-i :port</code> shows what is using a port

TABLE 26

Declaring and Typing Variables

`declare` (synonym: `typeset`) gives variables attributes that control their behavior: integer arithmetic, case conversion, export, and read-only status. Using `local` inside functions prevents variable leakage. `readonly` creates a constant that cannot be reassigned or unset.

Command	Example	Description
declare -i	<code>declare -i count=0</code>	- Declare variable as integer - arithmetic evaluated on assignment
declare -r	<code>declare -r PI=3.14</code>	- Declare read-only variable (constant) - cannot be reassigned or unset
readonly	<code>readonly MAX_RETRIES=3</code>	- Mark variable as read-only - equivalent to <code>declare -r</code>
local	<code>local tmp="value"</code>	- Declare function-scoped variable — only valid inside the function - does not leak to global scope
declare -x	<code>declare -x PATH</code>	Export variable to child processes - equivalent to <code>export</code>
declare -u	<code>declare -u upper</code>	Variable is auto-uppercased on assignment (Bash 4.0+)
declare -l	<code>declare -l lower</code>	Variable is auto-lowercased on assignment (Bash 4.0+)
declare -p	<code>declare -p varname</code>	Print variable attributes and value in a form that can be re-evaluated
declare -n	<code>declare -n ref=other_var</code>	Create a nameref — variable acts as an alias to another variable (Bash 4.3+)

References

Official Documentation

1. GNU Bash Reference Manual — Full Manual
2. GNU Bash Reference Manual — Redirections
3. GNU Bash Reference Manual — Command Substitution
4. GNU Bash Reference Manual — Command Grouping
5. GNU Bash Reference Manual — Shell Parameter Expansion
6. GNU Bash Reference Manual — Pattern Matching
7. GNU Bash Reference Manual — Bash Builtins
8. GNU Bash Reference Manual — Bourne Shell Builtins
9. GNU Bash Reference Manual — Looping Constructs
10. GNU Bash Reference Manual — Conditional Constructs
11. GNU Bash Reference Manual — Conditional Expressions
12. GNU Bash Reference Manual — Arrays
13. GNU Bash Reference Manual — Process Substitution
14. GNU Bash Reference Manual — The Set Builtin
15. GNU Bash Reference Manual — The Shopt Builtin
16. GNU Bash Reference Manual — Brace Expansion
17. GNU Bash Reference Manual — Directory Stack Builtins
18. GNU Bash Reference Manual — Arithmetic Expansion
19. GNU Bash Reference Manual — Special Parameters
20. GNU Bash Reference Manual — Bash Variables
21. GNU Bash Reference Manual — History Interaction
22. GNU Bash Reference Manual — Readline Killing Commands
23. GNU Bash Reference Manual — Pipelines
24. GNU Bash Reference Manual — Job Control Basics
25. GNU Bash Reference Manual — ANSI-C Quoting
26. GNU Coreutils — ls invocation
27. GNU Coreutils — cp invocation
28. GNU Coreutils — mv invocation
29. GNU Coreutils — rm invocation
30. GNU Coreutils — mkdir invocation
31. GNU Coreutils — ln invocation
32. GNU Coreutils — cat invocation
33. GNU Coreutils — head invocation
34. GNU Coreutils — tail invocation

- 35. GNU Coreutils — tac invocation
- 37. GNU Coreutils — sort invocation
- 39. GNU Coreutils — cut invocation
- 41. GNU Coreutils — wc invocation
- 43. GNU Coreutils — join invocation
- 45. GNU Coreutils — tee invocation
- 47. GNU Coreutils — chown invocation
- 49. GNU Coreutils — nohup invocation
- 51. GNU Coreutils — timeout invocation
- 53. GNU Coreutils — du invocation
- 55. GNU Coreutils — mktemp invocation
- 57. GNU Coreutils — uname invocation
- 59. GNU Coreutils — id invocation
- 61. GNU Coreutils — printenv invocation
- 63. GNU diffutils — Invoking diff
- 65. GNU sed Manual
- 67. GNU tar Manual
- 69. GNU wget Manual
- 71. GNU findutils Manual — find
- 73. Linux Manual Pages — tree(1)
- 75. Linux Manual Pages — kill(1)
- 77. Linux Manual Pages — renice(1)
- 79. Linux Manual Pages — ss(8)
- 81. Linux Manual Pages — ping(8)
- 83. Linux Manual Pages — rsync(1)
- 85. Linux Manual Pages — locate(1)
- 87. Linux Manual Pages — whereis(1)
- 89. Linux Manual Pages — more(1)
- 91. Linux Manual Pages — mkfifo(3)
- 93. Linux Manual Pages — pkill(1)
- 95. Linux Manual Pages — free(1)
- 97. Linux Manual Pages — hostname(1)
- 99. Linux Manual Pages — netstat(8)
- 101. OpenBSD man pages — scp(1)
- 103. curl Documentation — Man Page
- 105. bzip2 Manual
- 107. The Linux Documentation Project — Here Documents
- 109. The Linux Documentation Project — Parameter Substitution

Technical Blogs & Tutorials

- 111. DigitalOcean — How To Use cd, pwd, and ls
- 113. DigitalOcean — How To Use ps, kill, and nice
- 115. DigitalOcean — How to Set Permissions in Linux
- 117. DigitalOcean — Advanced Bash Scripting: Process Substitution and Traps
- 119. Red Hat — 8 fundamental Linux file-management commands
- 121. Red Hat — Five ways to use redirect operators in Bash
- 123. Red Hat — Navigating your filesystem in the Linux terminal
- 125. Red Hat — How to manipulate files with shell redirection and pipelines
- 127. Red Hat — How to check disk space on Linux
- 129. Baeldung — What Are the Special Dollar Sign Shell Variables?
- 131. Ryan's Tutorials — Learn Piping and Redirection
- 133. LinuxCommand.org — Learning the shell - Lesson 8: Expansion
- 135. SS64.com — Bash A-Z Index
- 137. SS64.com — How-To: Use Parenthesis/Brackets to group expressions
- 139. OSTechNix — Bash Expansion: The Complete Beginner's Guide
- 141. Warp Terminal — Bash Case Statement Syntax
- 143. Network World — Using bash options to change the behavior of scripts
- 145. OneUptime — How to Handle Process Substitution in Bash
- 147. OneUptime — How to Use trap for Signal Handling in Bash on Ubuntu
- 149. OneUptime — How to Handle Subshells in Bash Scripts

- 36. GNU Coreutils — nl invocation
- 38. GNU Coreutils — uniq invocation
- 40. GNU Coreutils — tr invocation
- 42. GNU Coreutils — paste invocation
- 44. GNU Coreutils — comm invocation
- 46. GNU Coreutils — chmod invocation
- 48. GNU Coreutils — chgrp invocation
- 50. GNU Coreutils — nice invocation
- 52. GNU Coreutils — df invocation
- 54. GNU Coreutils — stat invocation
- 56. GNU Coreutils — touch invocation
- 58. GNU Coreutils — date invocation
- 60. GNU Coreutils — whoami invocation
- 62. GNU Coreutils — env invocation
- 64. GNU grep Manual
- 66. GNU awk (gawk) Manual
- 68. GNU gzip Manual
- 70. GNU findutils Manual — xargs
- 72. Linux Manual Pages — glob(7)
- 74. Linux Manual Pages — ps(1)
- 76. Linux Manual Pages — top(1)
- 78. Linux Manual Pages — watch(1)
- 80. Linux Manual Pages — ip(8)
- 82. Linux Manual Pages — traceroute(8)
- 84. Linux Manual Pages — dig(1)
- 86. Linux Manual Pages — which(1)
- 88. Linux Manual Pages — less(1)
- 90. Linux Manual Pages — file(1)
- 92. Linux Manual Pages — signal(7)
- 94. Linux Manual Pages — killall(1)
- 96. Linux Manual Pages — uptime(1)
- 98. Linux Manual Pages — lsof(8)
- 100. OpenBSD man pages — ssh(1)
- 102. OpenBSD man pages — nc(1)
- 104. Zstandard (zstd) — Facebook Open Source
- 106. htop — Interactive Process Viewer
- 108. The Linux Documentation Project — Process Substitution
- 110. The Linux Documentation Project — Command Substitution

- 112. DigitalOcean — 50+ Essential Linux Commands
- 114. DigitalOcean — How To View and Update the Linux PATH
- 116. DigitalOcean — Shell Script if else
- 118. Red Hat — Linux Commands: jobs, bg, and fg
- 120. Red Hat — Linux environment variable tips and tricks
- 122. Red Hat — Linux Command Basics: 7 commands for process management
- 124. Red Hat — 5 ways to navigate the Linux terminal faster
- 126. Red Hat — Linux scripting: while loops in Bash
- 128. Baeldung — Here Document and Here String
- 130. Baeldung — Command Chaining in Bash
- 132. LinuxCommand.org — Learning the shell - Lesson 2: Navigation
- 134. LinuxCommand.org — Adventures with tput
- 136. SS64.com — How-To: Use Escape Characters, delimiters and Quotes
- 138. SS64.com — How-To: Command Substitution
- 140. Warp Terminal — How To Use Linux/Unix Wildcards
- 142. Warp Terminal — Bash printf Use Cases
- 144. Network World — Unix: Using pushd and popd for faster navigation
- 146. OneUptime — How to Handle Signal Trapping in Bash
- 148. OneUptime — How to Handle Conditional Statements in Bash
- 150. OneUptime — How to Handle Arrays in Bash Scripts

- 151. OneUptime — How to Handle Loops in Bash Scripts
- 153. OneUptime — How to Handle File Permissions with chmod and chown
- 155. OneUptime — Mastering Bash Parameter Expansion
- 157. Linux Journal — The Bash Trap Command
- 159. Linux Journal — Pattern Matching In Bash
- 161. Linux Journal — Work the Shell - Special Variables I: the Basics
- 163. Linux Journal — Simplifying Linux File Compression With Tar and Gzip

- 165. Linux Foundation — Classic SysAdmin: 14 tail and head commands

- 167. Opensource.com — Navigating the Bash shell with pushd and popd
- 169. Opensource.com — An introduction to parameter expansion in Bash

- 171. Contabo Blog — How to Use the Linux sed Command in 2026
- 173. CyberCiti.biz — How To Use Bash Parameter Substitution Like A Pro
- 175. sap1ens.com — Bash scripting best practices

- 177. Linux Handbook — Difference Between Single and Double Quote in Bash
- 179. Effective Shell — Mastering Conditional Logic
- 181. Shapedshed — Linux and Unix xargs command tutorial
- 183. phoenixNAP — Bash printf: How to Print a Variable in Bash
- 185. ArchWiki — Archiving and compression
- 187. ShShell.com — Data Refinement: sort, uniq, wc, cut, and tr
- 189. GeeksforGeeks — cut Command in Linux
- 191. GeeksforGeeks — Linux Permissions
- 193. HowToGeek — How to Compress and Extract Files Using tar
- 195. Medium — Viewing Files in Linux (cat, less, head, tail)
- 197. Medium — Archiving & Compression in Linux: Complete Guide

- 199. Robert Elder — Intro To 'mkfifo' Command In Linux
- 201. Stephan Max — Quick Directory Navigation in Bash with Builtins
- 203. GitHub — trinitib/Linux-Bash-Commands
- 205. MDN Web Docs — Command line crash course
- 207. TutorialsPoint — Here Document And Here String in Bash
- 209. Hostinger — How to use Bash array in scripting
- 211. Belief Driven Design — Shell Redirection 101
- 213. sysxplore — Bash 101: Indexed Arrays Explained

- 215. IOFlood — 'fg' Linux Command: Your Complete Guide to Job Control
- 217. LinuxConfig.org — Introduction to Bash shell redirections
- 219. CBT Nuggets — When to Use chmod vs chown
- 221. The Valley of Code — Process Management: bg - Background Jobs

- 223. TecMint — How to Use Redirection and Pipes in Linux
- 225. LabEx — How to Master Bash Subshell and Parentheses Techniques

- 152. OneUptime — How to Handle Here Documents in Bash
- 154. OneUptime — How to Handle String Manipulation in Bash
- 156. OneUptime — How to Handle Disk Space Analysis with du and df
- 158. Linux Journal — Bash's Built-in printf Function
- 160. Linux Journal — Efficient Text Processing in Linux: Awk, Cut, Paste
- 162. Linux Journal — Running Multiple Linux Commands Simultaneously
- 164. Linux Foundation — Classic SysAdmin: How to Kill a Process from the Command Line
- 166. Linux Foundation — Classic SysAdmin: How to Check Disk Space on Linux
- 168. Opensource.com — 7 Bash history shortcuts you will actually use
- 170. Opensource.com — How to program with Bash: Logical operators and shell expansions
- 172. Nick Janetakis — Difference between Double Ampersand and Semicolon
- 174. CyberCiti.biz — Understanding Linux/Unix umask value usage
- 176. DevOpsKit.dev — Linux File Permissions Explained: chmod, chown & Beyond
- 178. Linux Handbook — Special Variables in Bash Scripting
- 180. Cherry Servers — Top 20 Linux Network Commands
- 182. It's FOSS — Bash Basics #8: For, While and Until Loops
- 184. phoenixNAP — Bash case Statement: Syntax and Examples
- 186. ShShell.com — Reading the System: cat, less, head, and tail
- 188. GeeksforGeeks — Head Command in Linux With Examples
- 190. GeeksforGeeks — Chaining Commands in Linux
- 192. GeeksforGeeks — 25 Basic Linux Commands
- 194. HowToGeek — Essential Linux Text Editing Tools
- 196. Medium — Day 19: Text Processing with grep, awk & sed
- 198. Medium — Bash shell basics — pipes, redirection, named pipes, and coprocesses
- 200. temptemp3 — Using declare in bash
- 202. Thoughtbot — Input/Output Redirection in the Shell
- 204. GitHub Gist — Bash keyboard shortcuts
- 206. TutorialsPoint — Unix / Linux - Special Variables
- 208. Cornell CVW — An Introduction to Linux - Shells - I/O and Redirection
- 210. Belief Driven Design — Using Indexed and Associative Arrays in Bash
- 212. sysxplore — What are Subshells in Linux
- 214. Teaching HealthTech DTU — Understanding network and remote servers
- 216. NameHero — When to Use Here-String vs Here-doc in Bash
- 218. YOUStable — Chown Command in Linux Explained With Examples
- 220. Stefan Judis — Navigate your shell history with CTRL keys
- 222. KodeKloud — Linux - Create, Delete, Copy, and Move Files and Directories
- 224. Putorius — Linux tee Command Usage Examples
- 226. IONOS — An overview of the most important Linux commands

GitHub Repositories & Code Examples

- 227. GitHub — trinitib/Linux-Bash-Commands — comprehensive command reference
- 228. GitHub — dylanaraps/pure-bash-bible — pure bash alternatives to external processes

Video Resources

- 229. YouTube — The Complete Bash Scripting Course (Dave Eddy / ysap.sh)
- 231. YouTube — Process Substitution — Bash Scripting Course (pt25)
- 233. YouTube — Linux df and du Commands
- 235. YouTube — The Bash Trap Command — Responding to unix signals
- 230. YouTube — Linux Full Course for Beginners 2026
- 232. YouTube — Understanding Linux Named Pipes
- 234. YouTube — How To Use The Shell Commands 'pushd', 'popd' and 'dirs'

Industry Best Practice Guides & Books

- 236. Bash Scripting Best Practices — sap1ens.com
- 237. Google Shell Style Guide
- 238. ShellCheck — A static analysis tool for shell scripts